

Lab Exercise 12: 0-1 Knapsack Optimization using Dynamic Programming

Objective: To implement a program that maximizes total item value within a weight capacity limit using 0-1 Knapsack optimization.

Aim: To solve multi-option allocation problems using Dynamic Programming tables in $O(N \cdot W)$ time, avoiding exponential $O(2^N)$ recursive branches.

Theory: The 0-1 Knapsack problem exhibits overlapping subproblems and optimal substructure characteristics. A 2D dynamic programming array tracks maximum values for combinations of items and weights. For each item, the algorithm decides whether to skip it or include it by checking its weight against the remaining capacity.

Algorithm / Procedure:

1. Input: Number of items N , array vectors for values and weights, and maximum capacity weight W .
2. Initialize a 2D dynamic programming table dp of dimensions $(N + 1) \times (W + 1)$ with zero values.
3. Loop through each item i from 1 to N , and each capacity step w from 1 to W .
4. If the item's weight is less than or equal to w , calculate the maximum value choice:

$$dp[i][w] = \max(val[i-1] + dp[i-1][w - wt[i-1]], dp[i-1][w]).$$

Otherwise, default to skipping the item: $dp[i][w] = dp[i-1][w]$.

5. Output the value at $dp[N][W]$.

Program Code: Problem8_KnapsackOptimization.java

```
import java.util.*;
```

```
public class Problem8_KnapsackOptimization {  
  
    public static int maximizeKnapsackValue(int[] wt, int[] val, int W, int N) {  
  
        // TODO: Implement bottom-up 2D DP matrix loop here  
  
        return 0; // placeholder  
  
    }  
  
}
```

```
public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);

System.out.print("Enter item count and knapsack max weight capacity: ");

int n = sc.nextInt(); int maxW = sc.nextInt();

int[] wt = new int[n]; int[] val = new int[n];

System.out.println("Enter values:"); for(int i=0; i<n; i++) val[i] = sc.nextInt();

System.out.println("Enter weights:"); for(int i=0; i<n; i++) wt[i] = sc.nextInt();


int maxProfit = maximizeKnapsackValue(wt, val, maxW, n);

System.out.println("Maximum value achievable: " + maxProfit);

sc.close();

}

}

```

Sample Input / Output:

Input:

3 50

60 100 120

10 20 30

Output:

Maximum value achievable: 220

Evaluation Criteria (Total Marks: 10):

- Program compiles and runs successfully (2 Marks)
- Proper implementation of the 2D Dynamic Programming matrix (3 Marks)
- Correct balance optimization logic (2 Marks)
- Handling edge cases correctly (e.g., zero weight or zero value inputs) (2 Marks)
- Clear code formatting and inline comments (1 Mark)